



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)

ОТЧЁТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ
Проектно-технологическая практика

приказ Университета о направлении на практику от «24» марта 2026 г. № 2695-С

Отчет представлен к
рассмотрению:
Студент группы ИМБО-02-22

«18» апреля 2026 г.


Ким К.С.
(подпись и расшифровка подписи)

Отчет утвержден.
Допущен к защите:

Руководитель практики
от кафедры

«18» апреля 2026 г.


Даева С.Г.
(подпись и расшифровка подписи)

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА ПРОИЗВОДСТВЕННУЮ ПРАКТИКУ
Проектно-технологическая практика

Студенту 4 курса учебной группы ИМБО-02-22
Киму Кириллу Сергеевичу

Место и время практики: РТУ МИРЭА кафедра ПМ, с 23 марта 2026 г. по 18 апреля 2026 г.

Должность на практике: студент

1. СОДЕРЖАНИЕ ПРАКТИКИ:

1.1. Изучить: методы и алгоритмы анализа данных, применимые к решению поставленной задачи ВКР, инструментальные средства разработки и обработки данных (библиотеки, фреймворки, языки программирования).

1.2. Практически выполнить: разработать математическую модель или алгоритм решения задачи, подготовить и преобразовать данные (очистка, преобразование, формирование признаков), реализовать алгоритм или модель с использованием выбранных инструментальных средств, провести настройку параметров и отладку модели, описать структуру и логику работы разработанного решения.

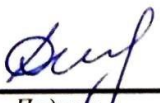
1.3. Ознакомиться: с современными программными средствами и технологиями анализа данных, машинного обучения и обработки данных.

2. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ: нет.

3. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: в процессе практики рекомендуется использовать периодические издания и отраслевую литературу годом издания не старше 5 лет от даты начала прохождения практики

Руководитель практики от кафедры
«23» марта 2026 г.

Задание получил
«23» марта 2026 г.


Подпись (Даева С.Г.)


Подпись (Ким К.С.)

СОГЛАСОВАНО:

Заведующий кафедрой:

«23» марта 2026 г.



Подпись (Саратова Т.Е.)

Проведенные инструктажи:

Охрана труда:

«23» марта 2026 г.

Инструктирующий



Подпись

Саратова Т.Е., заведующий
кафедрой ПМ

Инструктируемый



Подпись

Ким К.С.

Техника безопасности:

«23» марта 2026 г.

Инструктирующий



Подпись

Саратова Т.Е., заведующий
кафедрой ПМ

Инструктируемый



Подпись

Ким К.С.

Пожарная безопасность:

«23» марта 2026 г..

Инструктирующий



Подпись

Саратова Т.Е., заведующий
кафедрой ПМ

Инструктируемый

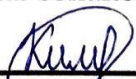


Подпись

Ким К.С.

С правилами внутреннего распорядка ознакомлен:

«23» марта 2026 г.



Подпись

Ким К.С.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

**РАБОЧИЙ ГРАФИК
ПРОВЕДЕНИЯ ПРОИЗВОДСТВЕННОЙ ПРАКТИКИ**

студента Кима К.С. 4 курса группы ИМБО-02-22 очной формы обучения, обучающегося по направлению подготовки 01.03.04 Прикладная математика, профиль «Анализ данных».

Неделя	Сроки выполнения	Этап	Отметка о выполнении
1	23.03.2026	Подготовительный этап, включающий в себя организационное собрание (Вводная лекция о порядке организации и прохождения производственной практики, инструктаж по технике безопасности, получение задания на практику)	<i>Дей</i>
1	23.03.2026– 28.03.2026	Проектно-технологический этап I: выбор и обоснование методов и алгоритмов анализа данных; выбор инструментальных средств (языков, библиотек); подготовка и анализ исходных данных; разработка структуры модели или алгоритма	<i>Дей</i>
2	30.03.2026	Представление руководителю материалов этапа I	<i>Дей</i>
2	30.03.2026– 04.04.2026	Проектно-технологический этап II: реализация модели или алгоритма; предобработка данных (очистка, преобразование, формирование признаков); настройка параметров модели; отладка и тестирование	<i>Дей</i>
3	06.04.2026	Представление руководителю материалов этапа II и доработанных материалов предыдущих этапов	<i>Дей</i>
3	06.04.2026– 11.04.2026	Проектно-технологический этап III: проведение экспериментальных исследований; оценка качества модели (метрики); анализ результатов и сравнение с существующими решениями	<i>Дей</i>
4	13.04.2026	Представление руководителю всей главы отчета по практике в соответствии с утвержденной темой ВКР	<i>Дей</i>
4	13.04.2026– 17.04.2026	Подготовка окончательной версии отчета по практике (Оформление материалов отчета в полном соответствии с требованиями на оформление письменных учебных работ студентов)	<i>Дей</i>

4	18.04.2026	Представление руководителю презентации по материалам исследовательского, аналитического и технологического этапов проектной и технологической (проектно-технологической) практик и окончательной версии отчета по практике руководителю в соответствии с указанными этапами технологической (проектно-технологической) практики, загруженного в СДО	
---	------------	---	---

Руководитель практики от
кафедры

 /Даева С.Г., к.ф.-м.н./

Обучающийся

 /Ким К.С./

Согласовано:

Заведующий кафедрой

 /Саратова Т.Е., д.т.н., доцент/

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	8
1.1 Описание реализации модели	8
1.1.1 Описание хода реализации	8
1.1.2 Реализация интерфейсов и взаимодействия компонентов	15
1.2 Анализ полученных результатов	16
1.2.1 Методология тестирования и валидации	16
1.2.2 Сравнительный анализ моделей и интерпретации	17
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСТОЧНИКОВ	23
ПРИЛОЖЕНИЯ	24

ВВЕДЕНИЕ

Футбол давно перестал быть исключительно спортивным зрелищем, а превратился в глобальную экономическую систему. Объем трансферных операций достиг больших масштабов: по данным FIFA, суммарные расходы на трансферы в пяти ведущих европейских лигах в 2023 году превысили 7,2 миллиарда евро. При этом принятие решений о стоимости игроков по-прежнему основывается на экспертных оценках и интуитивных суждениях, что вызывает существенные финансовые риски и приводит к неэффективному распределению ресурсов.

Целью практики является разработка модели оценки стоимости футбольных трансферов на основе игровой активности и физических характеристик игроков.

Задачи практики:

- выполнить сбор и очистку данных;
- спроектировать и рассчитать новые аналитические признаки;
- обучить модель;
- провести сравнительный анализ эффективности нескольких алгоритмов машинного обучения (линейных моделей и ансамблей деревьев решений);
- протестировать разработанную модель на реальных трансферах.

В этом разделе объектом исследования выступает процесс формирования трансферной стоимости профессиональных футболистов. Предметом исследования являются методы и алгоритмы машинного обучения, применимые для прогнозирования стоимости игрока. Практическая значимость работы заключается в создании автоматизированного инструмента, способного снизить субъективность при оценке футболистов и минимизировать финансовые риски клубов при заключении контрактов.

1 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

1.1 Описание реализации модели

Программная реализация модели выполнена на языке программирования Python в облачной среде Google Colaboratory [1.12].

Базовый стек технологий включает библиотеки:

- pandas — для загрузки, очистки и сложной агрегации табличных данных [1.10];
- numpy — для математических вычислений [1.9];
- scikit-learn — для построения моделей машинного обучения, предобработки данных и расчета метрик качества [1.11];
- matplotlib — для визуализации результатов [1.8];
- kagglehub — для автоматизированного скачивания актуальных датасетов.

Основная идея заключается в том, чтобы на основе характеристик футболиста (возраст, спортивная статистика, история травм, динамика рыночной стоимости) предсказать сумму трансфера.

1.1.1 Описание хода реализации

Для реализации модели необходимо:

1. Загрузка и обработка данных.

С помощью библиотеки kagglehub данные скачиваются автоматически (представлено в таблице 1.1). После скачивания загружаются несколько таблиц: история трансферов, профили игроков, рыночные стоимости, игровая статистика, травмы и выступления за сборную.

Таблица 1.1 — Характеристики исходных таблиц

Название файла	Количество строк	Количество колонок	Описание
transfer_history.csv	1 101 440	10	История трансферов игроков
player_profiles.csv	92 671	34	Профили игроков (дата рождения, позиция и др.)
player_market_value.csv	901 429	3	Динамика рыночной стоимости
player_performances.csv	1 878 719	20	Детальная игровая статистика по сезонам
player_injuries.csv	143 195	7	Данные о травмах
player_national_performances.csv	92 701	9	Статистика выступлений за сборную

2. Предобработка данных.

На этапе предобработки из таблицы истории трансферов были исключены бесплатные переходы, поскольку их нулевая стоимость искажает алгоритмы ценообразования машинного обучения.

Распределение стоимостей футболистов имеет сильную правостороннюю асимметрию (представлено на Рисунке 1.1), на футбольном рынке подавляющее большинство сделок совершается за небольшие суммы, тогда как переходы элитных игроков (суперзвезд) формируют большие выбросы. Для устранения асимметрии распределения целевой переменной применено логарифмическое преобразование.

$$y' = \ln(x + 1). \quad (1.1)$$

где x — фактическая сумма трансфера в евро, y' — логарифмированная целевая переменная.

Такое преобразование стабилизирует дисперсию, приближает распределение к нормальному и уменьшает влияние аномально больших значений (выбросов) на функцию потерь при обучении моделей.

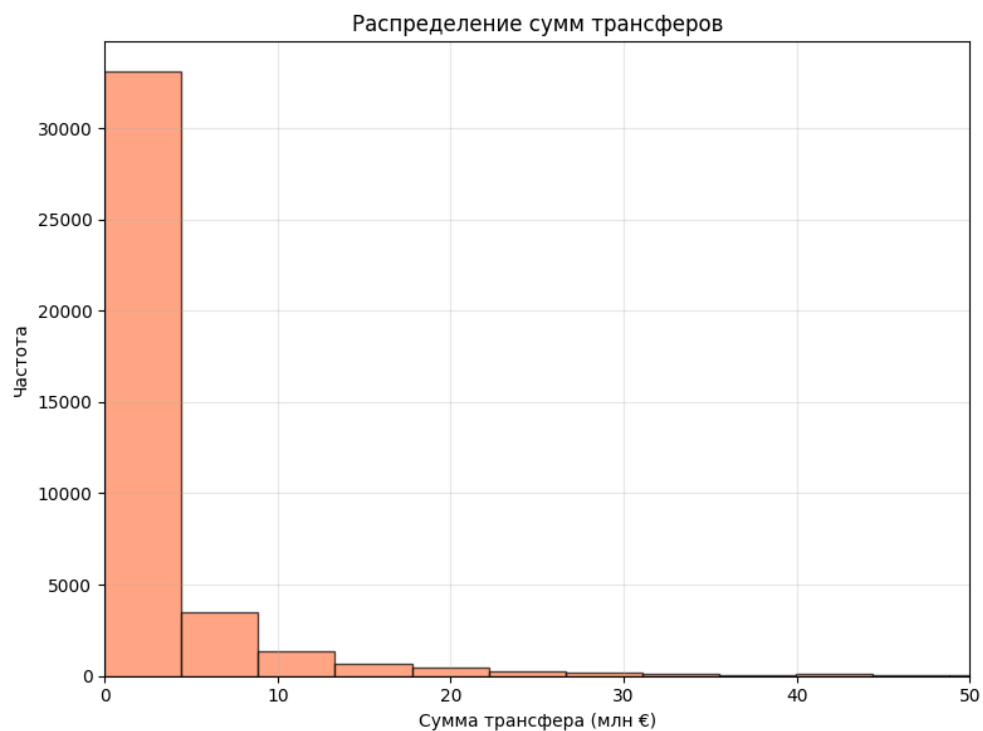


Рисунок 1.1 — Распределение трансферных сумм

Результат преобразования показан на Рисунке 1.2 — распределение стало близко к нормальному, что положительно влияет на устойчивость моделей к аномально большим значениям (выбросам). Проблема отсутствующих данных решена методом статистической импутации: пропуски в числовых признаках заполнены медианным значением, а в категориальных — модой.

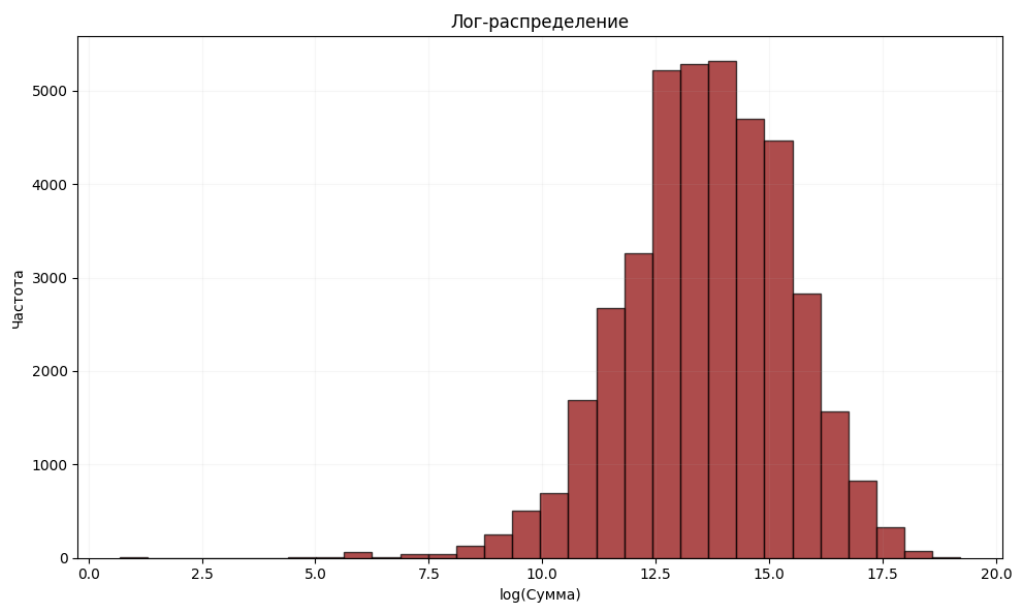


Рисунок 1.2 — Распределение трансферных сумм после логарифмирования

3. Формирование признаков.

Для построения прогнозной модели было сформировано 19 признаков, отражающих различные аспекты ценности игрока.

Для моделирования стоимости критически важен возраст игрока на момент совершения трансфера. Он рассчитывался как разность между датой трансфера и датой рождения игрока (с точностью до дня, затем перевод в годы). Поскольку зависимость рыночной стоимости от возраста нелинейна (стоимость быстро растёт у молодых игроков, достигает пика к 26–28 годам и постепенно снижается после 30 лет) в набор признаков был добавлен квадрат возраста. Это позволяет алгоритмам машинного обучения (особенно линейным моделям) учитывать параболическую динамику изменения стоимости на разных этапах карьеры игрока. На Рисунке 1.3 представлена средняя стоимость трансфера в зависимости от возрастной группы, что наглядно подтверждает описанный параболический тренд.

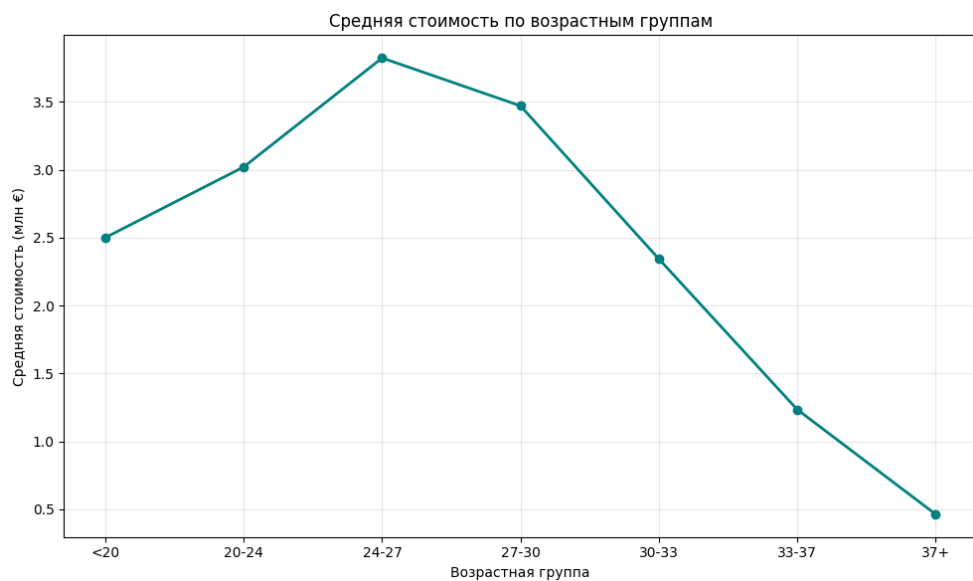


Рисунок 1.3 — Средняя стоимость по возрастным группам

4. Синхронизация временных рядов.

Экспертные оценки платформы Transfermarkt (`market_value`) представляют собой временной ряд: для каждого игрока известна дата обновления оценки и сама оценка. Дата трансфера и дата ближайшей предшествующей рыночной оценки почти никогда не совпадают в точности. Внутреннее соединение (INNER

JOIN) по дате невозможно, так как требует полного совпадения дат. Использование сдвига (например, берём оценку за прошлый месяц) было бы произвольным и могло бы привести к утечке будущей информации.

Поэтому экспертные оценки были присоединены к основному массиву данных с помощью функции `pd.merge_asof`. Поскольку даты обновления рыночной стоимости и даты фактических трансферов редко совпадают, использовался поиск ближайшей предшествующей оценки в пределах 365 дней [1.10]. Параметр `direction = «backward»` обеспечивает выбор оценки, которая была актуальна непосредственно перед трансфером. Для записей, у которых такая оценка отсутствовала, пропуски заполнялись медианным значением, но с учетом позиции и возрастной группы игрока. Это позволило сохранить все наблюдения и повысить репрезентативность данных.

5. Агрегация игровой статистики.

Разработан алгоритм, собирающий статистику игрока за два полных сезона, предшествующих дате трансфера. В агрегацию вошли показатели (голы, передачи, минуты, карточки и т.д.). Добавляются другие метрики:

- результативность за 90 минут считается по формуле:

$$goalsassistsper90 = \frac{goals + assists}{\left(\frac{minutes}{90}\right) + 10^{-5}}. \quad (1.2)$$

где *goals* — количество всех голов, *assists* — количество голевых передач, *minutes* — количество сыгранных минут в матче.

- голы без учета пенальти:

$$nonpenaltygoals = goals - penaltygoals. \quad (1.3)$$

где *goals* — общее количество голов, *penaltygoals* — количество голов с пенальти.

- коэффициент «сухих» матчей для вратарей и защитников:

$$clsheetratio = \frac{clsheet}{goalsconceded + 1}. \quad (1.4)$$

где *clsheet* — количество матчей, в которых команда не пропустила ни одного гола, *goalsconceded* — общее количество голов, которые команда позволила сопернику забить.

Также интегрированы данные по травмам (суммарное количество пропущенных дней до момента совершения трансфера), этот признак отражает физическую надежность игрока. По выступлениям за сборную для каждого игрока агрегированы голы, забитые за национальную команду (суммарно за всю карьеру). Данные о матчах за сборную также были добавлены, но их влияние оказалось менее значимым.

6. Разделение данных по времени и масштабирование.

Данные разделены по временному принципу: обучающая выборка — трансферы с 1973 по 2021 год (30 447 записей), тестовая — с 2022 по 2026 год (9 550 записей). Для корректной работы алгоритмов, чувствительных к масштабу переменных (линейные модели), выполнено масштабирование числовых признаков с помощью *StandardScaler*. Этот метод трансформирует данные так, чтобы их среднее значение было равно нулю, а стандартное отклонение — единице. Древовидные ансамбли обучались на немасштабированных данных, так как они инвариантны к монотонным преобразованиям признакового пространства.

7. Обучение и сравнение моделей.

Обучено четыре модели машинного обучения. Обучение моделей выполнялось с использованием зафиксированных гиперпараметров. Гиперпараметры каждой модели приведены в таблице 1.2.

Таблица 1.2 — Гиперпараметры обученных моделей

Модель	Описание
Ridge (L2-регуляризация)	$\alpha = 1$, max_iter = 1000, random_state = 42
Lasso (L1-регуляризация)	$\alpha = 0,001$, max_iter = 1000, random_state = 42
Random Forest	n_estimators = 200, max_depth = 10, min_samples_split = 10, min_samples_leaf = 5, random_state = 42, n_jobs = -1
Gradient Boosting	n_estimators = 150, learning_rate = 0,05, max_depth = 6, subsample = 0,8, random_state = 42

- Ridge (L2-регуляризация) добавляет квадратичный штраф на коэффициенты, снижая переобучение. Параметр регуляризации $\alpha = 1$ подобран экспериментально: при меньших значениях модель переобучалась, при больших — недообучалась.
- Lasso (L1-регуляризация) использует модульный штраф, способный обнулять незначимые коэффициенты, выполняя отбор признаков. $\alpha = 0,001$ — небольшое значение, позволяющее сохранить большинство признаков, но всё же оказывающее регуляризующий эффект;
- Random Forest строит 200 деревьев решений, каждое дерево строится на случайной подвыборке данных и случайном подмножестве признаков, а итоговый прогноз получается усреднением, максимальная глубина 10 ограничивает глубину дерева, предотвращая переобучение на шумах. [1.1];
- Gradient Boosting последовательно строит деревья, каждое следующее исправляет ошибки предыдущего. Скорость обучения 0,05, модель строит 150 деревьев. `subsample = 0,8` означает, что каждое дерево обучается на 80% случайно выбранных данных, что дополнительно предотвращает переобучение [1.5].

Процесс обучения показал, что линейные модели обучились быстро (меньше чем за 100 итераций), что объясняется выпуклостью целевой функции и небольшим количеством признаков. Однако их итоговая предсказательная способность на тестовой выборке оказалась отрицательной ($R^2 < 0$) для Ridge и для Lasso. Это свидетельствует о том, что зависимость между характеристиками игрока и его трансферной стоимостью носит нелинейный характер, который линейные модели не могут подобрать.

Построение ансамблевых методов потребовало большего вычислительного времени, но в результате ансамблевые методы показали высокие метрики. Случайный лес достиг $R^2 = 0,7837$. Градиентный бустинг показал чуть низкий результат $R^2 = 0,7763$. Таким образом, наилучшей моделью можно считать случайный лес [1.6].

1.1.2 Реализация интерфейсов и взаимодействия компонентов

Разработанная система не требует создания графического интерфейса (GUI), поскольку представляет собой аналитический конвейер [1.4], предназначенный для работы с данными. Взаимодействие компонентов системы осуществляется посредством передачи и трансформации структур данных `pandas.DataFrame`.

Основные компоненты:

- модуль загрузки — функции `load_csv()`, `extract_season_year()`, `get_age_group()`, `calculate_advanced_metrics()`. Отвечает за чтение CSV-файлов, извлечение года из названия сезона (например, обрезку строковых значений вида «2020/21» до временного формата 2020), группировку возраста и расчёт продвинутых метрик. Взаимодействие с данными осуществляется через метод `apply()` библиотеки `pandas` [1.10];
- модуль предобработки — очистка данных (удаление бесплатных трансферов), логарифмическое преобразование целевой переменной, заполнение пропусков медианой и модой;
- модуль генерации признаков — расчёт возраста и квадрата возраста, интеграция рыночной стоимости, агрегация статистики за 2 сезона, вычисление суммарного количества дней, пропущенных из-за травм, и агрегация статистики выступлений за национальную сборную (общее количество забитых мячей на международном уровне);
- модуль обучения и оценки — масштабирование признаков, обучение моделей, расчёт метрик и выбор лучшей модели по R^2 ;
- модуль анализа — визуализация оценки важности с помощью встроенной важности (feature importance) и permutation importance, прогноз для конкретного игрока.

Взаимодействие компонентов реализовано через вызов функций из основного скрипта. На вход принимает `DataFrame` и возвращает

преобразованный DataFrame. Для воспроизводимости результатов фиксирован `random_state = 42`. Полный листинг кода приведён в приложении А.

1.2 Анализ полученных результатов

1.2.1 Методология тестирования и валидации

Для оценки качества модели использовалась временная валидация — для прогнозных задач с хронологическими данными, исключающая утечку будущей информации. Обучающая выборка включала трансферы до 2021 года (1973–2021) более 30 тыс. записей, тестовая — с 2022 по 2026 год 9550 записей [1.2].

После обучения моделей был проведен сравнительный анализ их качества. Для оценки использовались следующие метрики:

- средняя абсолютная ошибка (MAE) — средняя абсолютная ошибка в евро. Показывает, на сколько в среднем прогноз отклоняется от фактической стоимости. Преимущество MAE — устойчивость к выбросам;

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (1.5)$$

- коэффициент детерминации (R^2) — доля дисперсии целевой переменной, объяснённая моделью. Значение $R^2 > 0$ означает, что модель даёт информацию о разбросе цен; $R^2 = 1$ соответствует идеальному прогнозу;

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (1.6)$$

- средняя абсолютная процентная ошибка (MAPE) — вычисляется как среднее абсолютных относительных отклонений, умноженное на 100%. Позволяет оценить точность в процентах и удобна для сравнения с другими исследованиями.

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|, \quad (1.7)$$

Все метрики вычислялись на тестовой выборке с использованием библиотеки `sklearn.metrics`.

Для контроля переобучения были рассчитаны метрики на обучающей выборке и на тестовой выборке представлено в таблице 1.3.

Таблица 1.3 — Метрики на обучающей и тестовой выборках (Random Forest)

Метрика	Обучающая выборка	Тестовая выборка
MAE	1 307 607 евро	1 755 481 евро
R ²	0,7643	0,7837
MAPE	13,0%	50,7%

Разница менее 0,03 по R² свидетельствует об отсутствии переобучения. На тестовой выборке присутствуют объекты с очень малыми фактическими ценами, из-за чего MAPE оказалась огромной.

1.2.2 Сравнительный анализ моделей и интерпретации

Сравнение метрик между собой представлено на Рисунке 1.4.

```

Обучение: Ridge (L2)
  Train: MAE = €2,794,550, R2 = -70.3750, MAPE = 32.3%
  Test : MAE = €3,999,126, R2 = -130.0120, MAPE = 43.5%

Обучение: Lasso (L1)
  Train: MAE = €2,748,704, R2 = -65.0703, MAPE = 33.1%
  Test : MAE = €3,875,680, R2 = -109.6928, MAPE = 42.9%

Обучение: Random Forest
  Train: MAE = €1,307,607, R2 = 0.7643, MAPE = 13.0%
  Test : MAE = €1,755,481, R2 = 0.7837, MAPE = 50.7%

Обучение: Gradient Boosting
  Train: MAE = €1,359,944, R2 = 0.7566, MAPE = 12.8%
  Test : MAE = €1,761,442, R2 = 0.7763, MAPE = 54.0%
```

Рисунок 1.4 — Сравнение метрик

Лучшие показатели продемонстрировал случайный лес (R² = 0,7837), показывая, что модель объясняет почти 78% разброса трансферных сумм [1.6].

При этом средняя абсолютная ошибка (MAE) составила около 1,75 млн евро, а средняя относительная ошибка (MAPE) остановилась на уровне 50,7%.

Линейные модели (Ridge и Lasso) показали отрицательные значения коэффициента R^2 [1.4]. Это объясняется тем, что зависимость между характеристиками игрока и его стоимостью носит нелинейный характер, которую способны эффективно улавливать только ансамблевые методы [1.3]. Например, совместное влияние возраста и травм (травма в молодости менее критична, чем в возрасте после 30 лет) автоматически учитывается деревьями решений.

Для интерпретации работы были проведены два анализа важности [1.7]. Анализ встроенной важности случайного леса (feature_importance) показал, что главным предиктором является предшествующая рыночная стоимость (37,8%) (представлено на Рисунке 1.5), а также её логарифм (36,6%). Далее следуют: голы за сборную (6,7%), жёлтые карточки (3,7%), и сыгранные минуты (2,7%). Интересно, что возрастные признаки оказались менее значимыми (в сумме около 2%), что может объясняться тем, что их влияние уже учтено в рыночной стоимости.

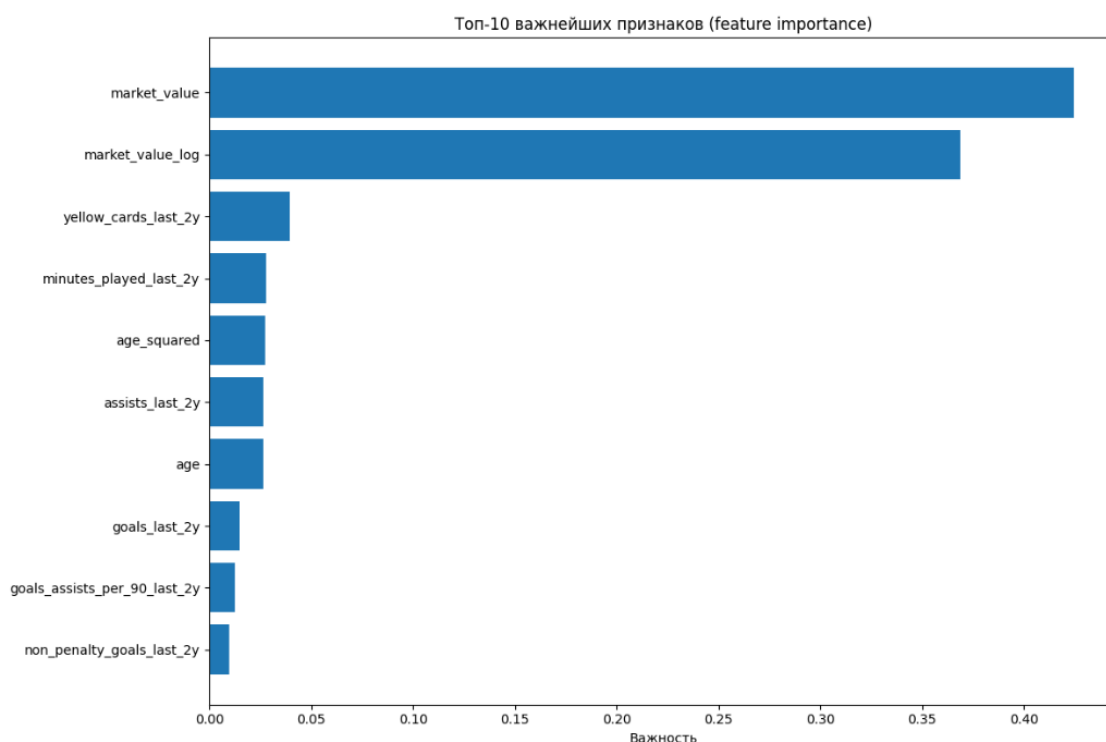


Рисунок 1.5 — Анализ важных признаков

Метод перестановочной важности (permutation importance) подтвердил лидирующую позицию рыночной стоимости (представлено на Рисунке 1.6). Кроме того, данный метод вывел в топ значимых факторов возраст и квадрат возраста. Это математически доказывает изначальную гипотезу о высокой важности нелинейной зависимости в связке «возраст — стоимость», которую алгоритм использует для корректировки базовой оценки игрока.

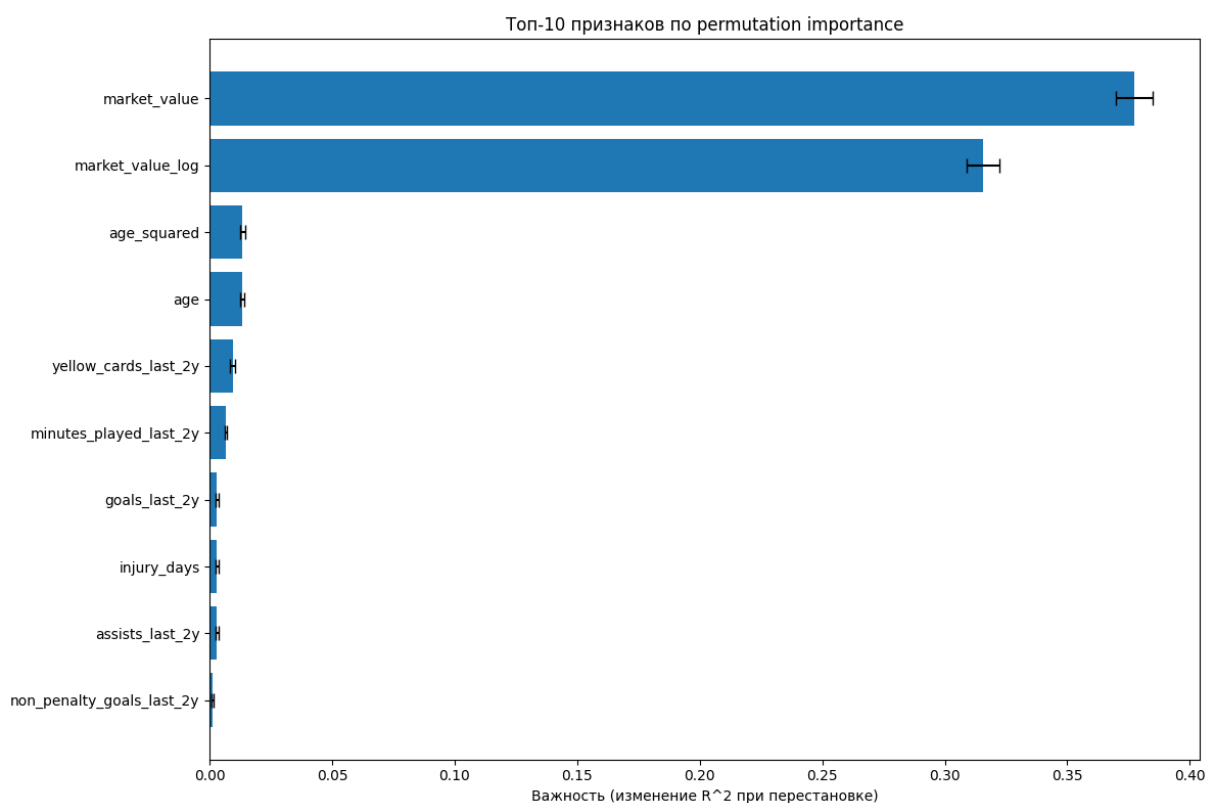


Рисунок 1.6 — Анализ важных признаков по permutation

Для проверки практической применимости модели были проанализированы трансферы известных футболистов. Относительная ошибка прогноза для каждого конкретного трансфера рассчитывалась по формуле:

$$\text{Ошибка} = \left(\frac{|\text{факт} - \text{прогноз}|}{\text{факт}} \right) * 100\%. \quad (1.8)$$

где факт — фактическая стоимость трансфера, прогноз — предсказанное значение моделью. Данная метрика показывает, на сколько процентов прогноз приближен к реальной сумме.

Был проведен анализ шести известных футболистов (Бензема, Ибрагимович, Гризмани, Суарес, Аслани, Арнау Тенас). Модель показала

относительные ошибки от 2,5% до 14,5%, что подтверждает практическую применимость модели.

Результаты сравнения представлены в таблице 1.4.

Таблица 1.4 — Результаты прогноза для конкретных трансферов

Игрок	Сезон	Фактическая стоимость	Прогноз	Ошибка
Карим Бензема	2009	35 млн. евро	31,7 млн. евро	9,4%
Златан Ибрагимович	2011	24 млн. евро	27,0 млн. евро	12,5%
Антуан Гризманн	2019	120 млн. евро	117,0 млн. евро	2,5%
Луис Суарес	2022	10 млн. евро	9,5 млн. евро	5,5%
Кристьян Аслани	2023	15,7 млн. евро	13,4 млн. евро	14,5%
Арнау Тенас	2025	2,5 млн. евро	2,6 млн. евро	5,8%

Для каждого из шести рассмотренных трансферов был проведён анализ причин отклонения прогнозной стоимости от фактической.

- Кристьян Аслани — модель ошиблась на 14,5%, недооценив игрока. Причина в том, что он молодой (21 год), маленькая игровая практика (1436 минут за два сезона). Рынок разглядел в нём потенциал и заплатил выше.
- Карим Бензема — модель снова оценила сдержанно, опираясь только на статистику, а клубы часто переплачивают за потенциал молодых футболистов.
- Златан Ибрагимович — модель переоценила игрока, поскольку трансфер происходил на фоне конфликта игрока с тренером в «Милане», клуб вынужден был продать игрока дешевле рыночной цены. Такие субъективные факторы в данных отсутствуют.
- Антуан Гризманн — этот игрок на пике карьеры, у него стабильная статистика и не было серьёзных травм. Доказывает, что модель корректно оценивает дорогостоящий трансфер.
- Луис Суарес — модель слегка занизила стоимость из-за возраста. Несмотря на выдающуюся карьеру в прошлом у этого игрока упало игровое время в последние сезоны перед трансфером.

- Арнау Тенас — модель переоценила игрока. Это связано с маленьким объёмом данных по вратарям и их показателям (сухие матчи и отражённые удары).

Таким образом, разработанная модель демонстрирует высокую предсказательную точность и может служить инструментом для предварительной оценки трансферной стоимости. Её преимущества — учёт широкого набора признаков, автоматическое выявление нелинейных зависимостей и использование только открытых данных. Дальнейшее улучшение возможно за счёт включения более детальной информации о контрактах, популярности в соцсетях и экономических показателях клубов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проектно-технологической практики была достигнута поставленная цель — разработана модель оценки стоимости футбольных трансферов на основе игровой активности и физических характеристик игроков.

В технологическом разделе была выполнена практическая реализация в среде Google Colaboratory. Реализован полный конвейер обработки данных. Внедрен принцип хронологического разделения выборки, что исключило риск переобучения. Сравнительное тестирование четырех алгоритмов машинного обучения доказало подавляющее превосходство ансамблевых методов. Алгоритм случайного леса продемонстрировал наилучшие метрики качества. С помощью анализа важности признаков было математически доказано определяющее влияние возраста и недавней игровой практики на итоговую стоимость игрока.

Практическая применимость модели подтверждена на реальных трансферах известных футболистов: относительная ошибка прогноза составила от 2,5% до 14,5%. Модель может служить инструментом для оценки трансферной стоимости, позволяя клубам и аналитическим агентствам снижать финансовые риски и принимать более обоснованные решения.

Все поставленные задачи были успешно выполнены.

СПИСОК ИСТОЧНИКОВ

1 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

- 1.1 Брейман Л. Случайные леса // Машинное обучение. — 2001. — Т. 45, № 1. — С. 5-32.
- 1.2 Вьюгин В. В. Математические основы теории машинного обучения и прогнозирования. — М.: МЦНМО, 2013. — 387 с.
- 1.3 Маккини У. Python и анализ данных. — М.: ДМК Пресс, 2020. — 540 с.
- 1.4 Мюллер А., Гвидо С. Введение в машинное обучение с помощью Python. — М.: Вильямс, 2017. — 480 с.
- 1.5 Фридман Дж. Жадная аппроксимация функций: машина градиентного бустинга // Анналы статистики. — 2001. — Т. 29, № 5. — С. 1189-1232.
- 1.6 Шолле Ф. Машинное обучение и глубокое обучение с Python, scikit-learn и TensorFlow 2. — СПб.: Питер, 2022. — 848 с.
- 1.7 Altmann A., Toloşi L., Sander O., Lengauer T. Permutation importance: a corrected feature importance measure // Bioinformatics. — 2010. — Vol. 26, № 10. — P. 1340-1347.
- 1.8 Документация библиотеки Matplotlib [Электронный ресурс]. — URL: <https://matplotlib.org/stable/> (дата обращения: 29.03.2026).
- 1.9 Документация библиотеки NumPy [Электронный ресурс]. — URL: <https://numpy.org/doc/> (дата обращения: 29.03.2026).
- 1.10 Документация библиотеки Pandas [Электронный ресурс]. — URL: <https://pandas.pydata.org/docs/> (дата обращения: 29.03.2026).
- 1.11 Документация библиотеки Scikit-learn [Электронный ресурс]. — URL: https://scikit-learn.org/stable/user_guide.html (дата обращения: 01.03.2026).
- 1.12 Среда разработки Google Colaboratory [Электронный ресурс]. — URL: <https://colab.research.google.com/> (дата обращения: 29.03.2026).

ПРИЛОЖЕНИЯ

Приложение А — Пример реализации кода

Приложение А

Пример реализации логики программы

Листинг А.1 — Импорт библиотек и загрузка данных

```
!pip install kagglehub -q
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, r2_score,
mean_absolute_percentage_error
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.inspection import permutation_importance
import kagglehub
print("Загрузка через kagglehub...")
path = kagglehub.dataset_download("xfkzujqjvx97n/football-datasets")
print(f"Данные загружены в: {path}")
def load_csv(file_path, description=""):
    full_path = os.path.join(path, file_path)
    df = pd.read_csv(full_path, low_memory=False)
    print(f"    {description:30}: {len(df):8,} строки, {len(df.columns):3d}
колонок")
    return df

transfer_history = load_csv('transfer_history/transfer_history.csv',
"История трансферов")
player_profiles = load_csv('player_profiles/player_profiles.csv', "Профили
игроков")
market_values = load_csv('player_market_value/player_market_value.csv',
"Рыночная стоимость")
player_performances =
load_csv('player_performances/player_performances.csv', "Выступления игроков")
player_injuries = load_csv('player_injuries/player_injuries.csv', "Травмы
игроков")
player_national =
load_csv('player_national_performances/player_national_performances.csv',
"Сборная")
```

Листинг A.2 — Вспомогательные функции (часть 1)

```
def extract_season_year(season):
    if pd.isna(season):
        return None
    s = str(season).strip()
    if '/' in s:
        first = s.split('/')[0]
        if len(first) == 4:
            return int(first)
        elif len(first) == 2:
            return 2000 + int(first) if int(first) < 50 else 1900 +
int(first)
    if s.isdigit() and len(s) == 4:
        return int(s)
    return None

def get_age_group(age):
    if pd.isna(age):
        return 'Unknown'
    if age < 20:
        return '<20'
    elif age < 24:
        return '20-24'
    elif age < 28:
        return '24-27'
    elif age < 31:
        return '27-30'
    elif age < 34:
        return '30-33'
    elif age < 37:
        return '33-37'
    else:
        return '37+'
```

Листинг A.3 — Вспомогательные функции (часть 2)

```
def calculate_advanced_metrics(df_perf):
    df = df_perf.copy()
    if 'goals' in df.columns and 'assists' in df.columns and
'minutes_played' in df.columns:
        df['goals_assists_per_90'] = (df['goals'] + df['assists']) /
(df['minutes_played'] / 90 + 1e-5)
        df['goals_assists_per_90'] =
df['goals_assists_per_90'].replace([np.inf, -np.inf], 0).fillna(0)
    if 'penalty_goals' in df.columns and 'goals' in df.columns:
        df['non_penalty_goals'] = df['goals'] -
df['penalty_goals'].fillna(0)
    if 'clean_sheets' in df.columns and 'goals_conceded' in df.columns:
        df['clean_sheet_ratio'] = df['clean_sheets'] /
(df['goals_conceded'] + 1)
    return df
```

Листинг А.4 — Очистка данных и расчёт возраста

```
df_transfers = transfer_history[transfer_history['transfer_fee'] >
0].copy()

df_transfers['log_fee'] = np.log1p(df_transfers['transfer_fee'])

df_transfers['season_year'] =
df_transfers['season_name'].apply(extract_season_year)

df_transfers = df_transfers.dropna(subset=['transfer_date', 'player_id'])

player_profiles['birth_date'] =
pd.to_datetime(player_profiles['date_of_birth'], errors='coerce')

df_transfers['transfer_date'] =
pd.to_datetime(df_transfers['transfer_date'], errors='coerce')

df = df_transfers.merge(
    player_profiles[['player_id', 'birth_date', 'position', 'height',
'foot']],
    on='player_id',
    how='left'
)

df['age'] = (df['transfer_date'] - df['birth_date']).dt.days / 365.25
df['age'] = df['age'].round(0)

age_median = df['age'].median()
df['age'] = df['age'].fillna(age_median)

df['age_squared'] = df['age'] ** 2
df['age_group'] = df['age'].apply(get_age_group)
```


Листинг А.5 — Разведочный анализ (часть 1)

```
fig, ax = plt.subplots(figsize=(12, 6))
# 5.1. Распределение по возрасту
age_bins = np.arange(15, 41, 2)
ax.hist(df['age'], bins=age_bins, color='blue', edgecolor='black',
alpha=0.7)
ax.axvline(x=df['age'].median(), color='red', linestyle='--', linewidth=2,
label=f"Медиана: {df['age'].median():.1f}")
ax.set_xlabel('Возраст (лет)')
ax.set_ylabel('Количество игроков')
ax.set_title('Распределение игроков по возрасту')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(10, 6))
# 5.2. Распределение по позициям
position_counts = df['position'].value_counts().head(10)
colors = plt.cm.Set3(np.linspace(0, 1, len(position_counts)))
ax.barh(position_counts.index, position_counts.values, color=colors)
ax.set_xlabel('Количество игроков')
ax.set_title('Распределение по позициям')
ax.grid(True, alpha=0.3, axis='x')
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(8, 6))
# 5.3. Распределение стоимости трансферов
fee_millions = df['transfer_fee'] / 1_000_000
ax.hist(fee_millions, bins=50, color='coral', edgecolor='black',
alpha=0.7)
ax.set_xlabel('Сумма трансфера (млн €)')
ax.set_ylabel('Частота')
ax.set_title('Распределение сумм трансферов')
ax.set_xlim([0, 50])
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Листинг А.6 — Разведочный анализ (часть 2)

```
fig, ax = plt.subplots(figsize=(10, 6))
# inset = ax.inset_axes([0.6, 0.5, 0.35, 0.4])
ax.hist(np.log1p(df['transfer_fee']), bins=30, color='darkred',
edgecolor='black', alpha=0.7)
ax.set_xlabel('log(Сумма)')
ax.set_ylabel('Частота')
ax.set_title('Лог-распределение')
ax.grid(True, alpha=0.1)
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(10, 6))
# 5.4. Средняя стоимость по возрастным группам
age_groups = ['<20', '20-24', '24-27', '27-30', '30-33', '33-37', '37+']
df['age_group_simple'] = pd.cut(df['age'], bins=[0, 20, 24, 27, 30, 33, 37,
50],
                                labels=age_groups)
age_group_avg = df.groupby('age_group_simple')['transfer_fee'].mean() /
1_000_000
age_group_avg.plot(kind='line', marker='o', ax=ax, color='teal',
linewidth=2)
ax.set_xlabel('Возрастная группа')
ax.set_ylabel('Средняя стоимость (млн €)')
ax.set_title('Средняя стоимость по возрастным группам')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Листинг А.7 — Разведочный анализ (часть 3)

```
fig, ax = plt.subplots(figsize=(10, 6))

# 5.5. Динамика стоимости по сезонам
season_stats = df.groupby('season_year').agg({
    'transfer_fee': ['mean', 'median', 'count']
}).round(0)
season_stats.columns = ['mean_fee', 'median_fee', 'count']
season_stats = season_stats[(season_stats.index >= 1985) &
(season_stats.index <= 2025)]

# Основной график стоимости
ax.plot(season_stats.index, season_stats['mean_fee'] / 1_000_000,
        marker='o', linewidth=2, markersize=6, color='blue',
label='Средняя стоимость')
ax.plot(season_stats.index, season_stats['median_fee'] / 1_000_000,
        marker='s', linewidth=2, markersize=6, color='red',
label='Медианная стоимость')
ax.set_xlabel('Сезон')
ax.set_ylabel('Стоимость (млн €)')
ax.set_title('Динамика стоимости трансферов по сезонам')
ax.grid(True, alpha=0.3)
ax.legend(loc='upper left')

# Вторая ось для количества трансферов
ax5_twin = ax.twinx()
ax5_twin.bar(season_stats.index, season_stats['count'], alpha=0.4,
color='lime',
            width=0.5, label='Количество трансферов')
ax5_twin.set_ylabel('Количество трансферов')
ax5_twin.legend(loc='upper right')
plt.tight_layout()
plt.show()
```

Листинг А.8 — Интеграция рыночной стоимости

```
market_values['date_unix'] = pd.to_datetime(market_values['date_unix'],
errors='coerce')

market_values = market_values[market_values['value'] > 0]

df = df.sort_values('transfer_date')
market_values = market_values.sort_values('date_unix')

df = pd.merge_asof(
    df,
    market_values[['player_id', 'date_unix', 'value']],
    left_on='transfer_date',
    right_on='date_unix',
    by='player_id',
    direction='backward',
    tolerance=pd.Timedelta('365D')
)

df = df.rename(columns={'value': 'market_value'})
df['has_market_value'] = df['market_value'].notna()
df['temp_age_group'] = pd.cut(df['age'], bins=[0, 22, 27, 32, 100],
labels=['young', 'prime', 'peak', 'veteran'])

market_value_medians = df[df['has_market_value']].groupby(['position',
'temp_age_group'])['market_value'].median().to_dict()

for idx, row in df[~df['has_market_value']].iterrows():
    key = (row['position'], row['temp_age_group'])
    if key in market_value_medians and not
pd.isna(market_value_medians[key]):
        df.loc[idx, 'market_value'] = market_value_medians[key]
    else:
        df.loc[idx, 'market_value'] =
df[df['has_market_value']]['market_value'].median()

df['market_value_log'] = np.log1p(df['market_value'])
df = df.drop(columns=['temp_age_group'])
```

Листинг А.9 — Игровая статистика за 2 года до трансфера (часть 1)

```
player_performances['season_year'] =
player_performances['season_name'].apply(extract_season_year)

ALL_STATS = [
    'goals',      'assists',      'minutes_played',      'yellow_cards',
    'direct_red_cards',      'second_yellow_cards',      'own_goals',      'penalty_goals',
    'goals_conceded', 'clean_sheets']

agg_dict = {}
for stat in ALL_STATS:
    if stat in player_performances.columns:
        agg_dict[stat] = 'sum'
    else:
        print(f"Колонка '{stat}' не найдена, пропускаем")

player_performances = calculate_advanced_metrics(player_performances)

advanced_metrics      =      ['goals_assists_per_90',      'non_penalty_goals',
'clean_sheet_ratio']
for metric in advanced_metrics:
    if metric in player_performances.columns:
        agg_dict[metric] = 'mean'
        print(f"Добавлена продвинутая метрика: {metric}")

perf_agg              =              player_performances.groupby(['player_id',
'season_year']).agg(agg_dict).reset_index()

stat_columns = list(agg_dict.keys())
```

Листинг A.10 — Игровая статистика за 2 года до трансфера (часть 2)

```
stats = []
missing_stats = 0

for idx, row in df.iterrows():
    if idx % 1000 == 0 and idx > 0:
        print(f"    Обработано {idx} трансферов")

    player_stats = perf_agg[
        (perf_agg['player_id'] == row['player_id']) &
        (perf_agg['season_year'] >= row['season_year'] - 2) &
        (perf_agg['season_year'] < row['season_year'])
    ]

    row_stats = {}
    if len(player_stats) == 0:
        missing_stats += 1
        for stat in stat_columns:
            row_stats[stat] = 0
    else:
        for stat in stat_columns:
            if stat in ['goals_assists_per_90', 'clean_sheet_ratio']:
                row_stats[stat] = player_stats[stat].mean()
            else:
                row_stats[stat] = player_stats[stat].sum()
    stats.append(row_stats)

stats_df = pd.DataFrame(stats)

for col in stats_df.columns:
    df[f'{col}_last_2y'] = stats_df[col]

    if col in ['goals', 'assists', 'minutes_played']:
        total = stats_df[col].sum()
        if col == 'minutes_played':
            print(f"    {col}_last_2y: {total:,.0f} минут")
        else:
            print(f"    {col}_last_2y: {total:,.0f}")
```

Листинг A.11 — Травмы и выступления за сборную

```
player_injuries['from_date']
pd.to_datetime(player_injuries['from_date'], errors='coerce')

injury_stats = []
for idx, row in df.iterrows():
    inj = player_injuries[
        (player_injuries['player_id'] == row['player_id']) &
        (player_injuries['from_date'] <= row['transfer_date'])
    ]
    injury_stats.append(inj['days_missed'].sum())

df['injury_days'] = injury_stats
agg_dict_national = {'goals': 'sum'}
for col in player_national.columns:
    if col not in ['player_id', 'season', 'season_name', 'date'] and col
not in agg_dict_national:
        if player_national[col].dtype in ['int64', 'float64']:
            agg_dict_national[col] = 'sum'
            print(f"Дополнительная колонка для агрегации: {col}")

national_agg
player_national.groupby('player_id').agg(agg_dict_national).reset_index()

rename_dict = {}
for col in agg_dict_national.keys():
    if col == 'goals':
        rename_dict[col] = 'national_goals'
    else:
        rename_dict[col] = f'national_{col}'

national_agg = national_agg.rename(columns=rename_dict)

df = df.merge(national_agg, on='player_id', how='left')

# Заполняем пропуски нулями
for col in rename_dict.values():
    if col in df.columns:
        df[col] = df[col].fillna(0)
```

Листинг A.12 — Финальный набор признаков и подготовка данных

```
performance_cols = [col for col in df.columns if col.endswith('_last_2y')]
df['position_simple'] = df['position'].fillna('Unknown').apply(
    lambda x: x.split('-')[0] if pd.notna(x) and '-' in str(x) else str(x)
)
FEATURES = [
    'age', 'age_squared', 'market_value',
    'market_value_log', 'injury_days', 'national_goals'
] + performance_cols

available_features = [f for f in FEATURES if f in df.columns]

df_model = df[available_features + ['log_fee', 'season_year',
'transfer_fee', 'position_simple']].copy()

for col in available_features:
    if col in df_model.columns:
        if df_model[col].dtype in ['int64', 'float64']:
            median_val = df_model[col].median()
            df_model[col] = df_model[col].fillna(median_val)
        else:
            mode_val = df_model[col].mode()[0] if not
df_model[col].mode().empty else 'Unknown'
            df_model[col] = df_model[col].fillna(mode_val)
```

Листинг A.13 — Разделение данных по времени и масштабирование

```
train = df[df['transfer_date'].dt.year <= 2021]
test = df[df['transfer_date'].dt.year > 2021]
X_train = train[available_features]
y_train = train['log_fee']
X_test = test[available_features]
y_test = test['log_fee']
y_train_real = np.expm1(y_train)
y_test_real = np.expm1(y_test)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```


Листинг A.14 — Обучение и сравнение моделей

```
models = {
    'Ridge (L2)': Ridge(alpha=1, random_state=42, max_iter=1000),
    'Lasso (L1)': Lasso(alpha=0.001, random_state=42, max_iter=1000),
    'Random Forest': RandomForestRegressor(n_estimators=200, max_depth=10,
min_samples_split=10, min_samples_leaf=5, random_state=42, n_jobs=-1),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=150,
learning_rate=0.05, max_depth=6, subsample=0.8, random_state=42)}

results_train = []
results_test = []
best_model = None
best_r2_test = -np.inf

for name, model in models.items():
    print(f"\nОбучение: {name}")
    if name in ['Ridge (L2)', 'Lasso (L1)']:
        model.fit(X_train_scaled, y_train)
        y_pred_train = model.predict(X_train_scaled)
        y_pred_test = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred_train = model.predict(X_train)
        y_pred_test = model.predict(X_test)
    y_pred_train_real = np.expml(y_pred_train)
    y_pred_test_real = np.expml(y_pred_test)

    mae_train = mean_absolute_error(y_train_real, y_pred_train_real)
    r2_train = r2_score(y_train_real, y_pred_train_real)
    mape_train = mean_absolute_percentage_error(y_train_real,
y_pred_train_real)
    results_train.append({'Модель': name, 'MAE_train (€)': mae_train,
'R^2_train': r2_train, 'MAPE_train (%)': mape_train})
    mae_test = mean_absolute_error(y_test_real, y_pred_test_real)
    r2_test = r2_score(y_test_real, y_pred_test_real)
    mape_test = mean_absolute_percentage_error(y_test_real,
y_pred_test_real)
    results_test.append({'Модель': name, 'MAE_test (€)': mae_test,
'R^2_test': r2_test, 'MAPE_test (%)': mape_test})
```

Листинг A.15 — Сводная таблица сравнения train/test

```
print(f"    Train: MAE = €{mae_train:,.0f}, R2 = {r2_train:.4f}, MAPE = {mape_train:.1f}%")

print(f"    Test : MAE = €{mae_test:,.0f}, R2 = {r2_test:.4f}, MAPE = {mape_test:.1f}%")

if r2_test > best_r2_test:
    best_r2_test = r2_test
    best_model = model
    best_model_name = name

# Сводная таблица сравнения train/test
print("Сравнение метрик на обучающей и тестовой выборках")

train_df = pd.DataFrame(results_train).sort_values('R^2_train', ascending=False)
test_df = pd.DataFrame(results_test).sort_values('R^2_test', ascending=False)

comparison = pd.merge(train_df, test_df, on='Модель', suffixes=('_train', '_test'))

print(comparison[['Модель', 'R^2_train', 'R^2_test', 'MAE_train (€)', 'MAE_test (€)']].to_string(index=False))

print(f"\nЛучшая модель: {best_model_name} (R2_test = {best_r2_test:.4f})")
```

Листинг A.16 — Анализ важности признаков

```
use_scaled = 'Ridge' in best_model_name or 'Lasso' in best_model_name
X_test_best = X_test_scaled if use_scaled else X_test

# Встроенная важность (для древовидных моделей)
if hasattr(best_model, 'feature_importances_'):
    importance = pd.DataFrame({
        'Признак': available_features,
        'Важность': best_model.feature_importances_
    }).sort_values('Важность', ascending=False)
    print(importance.head(10))

# Permutation importance
perm = permutation_importance(best_model, X_test_best, y_test,
                              n_repeats=10, random_state=42, scoring='r2',
n_jobs=-1)
perm_importance = pd.DataFrame({
    'Признак': available_features,
    'Важность': perm.importances_mean,
    'Std': perm.importances_std
}).sort_values('Важность', ascending=False)
print(perm_importance.head(10))
```

Листинг A.17 — Анализ конкретных трансферов (часть 1)

```
def find_player(player_name):
    if player_profiles.empty:
        print("    Данные профилей не загружены")
        return None

    matches = player_profiles[
        player_profiles['player_name'].str.contains(player_name,
case=False, na=False)
    ]

    if len(matches) == 0:
        print(f"    Игрок '{player_name}' не найден")
        return None

    print(f"    Найдено {len(matches)} совпадений:")
    for i, (_, row) in enumerate(matches.iterrows()):
        print(f"        {i+1}. {row['player_name']} (ID: {row['player_id']})")
    return matches.iloc[0]

def analyze_player_transfer(player_id, target_year):
    transfer = df[(df['player_id'] == player_id) & (df['season_year'] ==
target_year)]

    if len(transfer) == 0:
        print(f"    Трансфер {target_year} года не найден")
        player_transfers = df[df['player_id'] ==
player_id].sort_values('season_year')
        if len(player_transfers) > 0:
            print("    Доступные трансферы:")
            for _, t in player_transfers.iterrows():
                from_team = t.get('from_team_name', 'Unknown')
                to_team = t.get('to_team_name', 'Unknown')
                print(f"        • {t['season_year']}: {from_team} → {to_team}
(€{t['transfer_fee']:,.0f})")
            return None
```

Листинг А.18 — Анализ конкретных трансферов (часть 2)

```
data = transfer.iloc[0]

profile_data = {}
for feat in available_features:
    profile_data[feat] = data.get(feat, 0)

profile = pd.DataFrame([profile_data])[available_features]

if use_scaled:
    profile_scaled = scaler.transform(profile)
    pred_log = best_model.predict(profile_scaled)[0]
else:
    pred_log = best_model.predict(profile)[0]

pred_value = np.expml(pred_log)
actual = data['transfer_fee']

if actual > 0:
    error_abs = abs(actual - pred_value)
    error_pct = ((error_abs / actual) * 100)
else:
    error_abs = 0
    error_pct = 0
    player_info = player_profiles[player_profiles['player_id'] ==
player_id]
    player_name = player_info.iloc[0]['player_name'] if len(player_info) >
0 else "Неизвестно"
    print(f"Анализ трансфера: {player_name}")
    print(f"    Год трансфера:      {data['season_year']}")
    print(f"    Возраст:                {data['age']:.1f} лет")
    print(f"    Позиция:                  {data.get('position', 'Unknown')}")
    print(f"                Фактическая      сумма:      €{actual:,.0f}
(€{actual/1_000_000:.2f}M)")
    print(f"                Прогноз      модели:      €{pred_value:,.0f}
(€{pred_value/1_000_000:.2f}M)")
    print(f"    Абсолютная ошибка: €{error_abs:,.0f}")
    print(f"    Относительная ошибка: {error_pct:.1f}%")
```

Листинг A.19 — Анализ конкретных трансферов (часть 3)

```
players_to_analyze = [
    ("Karim Benzema", 2009),
    ("Zlatan Ibrahimović", 2011),
    ("Antoine Griezmann", 2019),
    ("Luis Suárez", 2022),
    ("Kristjan Asllani", 2023),
    ("Arnau Tenas", 2025)
]

case_studies = []

print("Анализ известных трансферов")

for player_name, year in players_to_analyze:
    print(f"\nПоиск: {player_name} ({year})")
    player = find_player(player_name)

    if player is not None:
        result = analyze_player_transfer(player['player_id'], year)
        if result is not None:
            case_studies.append(result)

case_df = pd.DataFrame(case_studies)
print("\nСводная таблица результатов анализа")

display_df = case_df.copy()
display_df['actual_fee'] = display_df['actual_fee'].apply(lambda x:
    f"€{x/1_000_000:.1f}M")
display_df['predicted_fee'] = display_df['predicted_fee'].apply(lambda x:
    f"€{x/1_000_000:.1f}M")
display_df['error'] = display_df['error'].apply(lambda x: f"{x:.1f}%")

print(display_df[['player_name', 'season', 'actual_fee', 'predicted_fee',
    'error']].to_string(index=False))
```